

TASKS OF DAILY

DAY01

TOPIC: LINUX COMMANDS

1. Create user with name Techie and provide sudo access to user.
 2. Navigate to the home directory.
 3. Create a new directory.
 4. List the contents of a directory.
 5. Change the current directory.
 6. Create a new empty file.
 7. View the contents of a file.
 8. Copy a file to another location.
 9. Move a file to another location.
 10. Rename a file.
 11. Delete a file.
 12. Grant or revoke permissions on a file or directory.
 13. View the current date and time.
 14. Check the system uptime.
 15. View the running processes.
 16. Kill a running process.
 17. Install a package using the package manager (e.g., apt or yum).
 18. Update the system packages.
 19. Create a symbolic link.
 20. Search for files using the find command.
 21. Compress and decompress files using tar.
 22. Monitor system resources with top or htop.
 23. Create and manage user groups.
 24. Set up SSH password less authentication.
 25. Monitor log files using tail or grep.
 26. Set up a web server (e.g., Apache or Nginx).
 27. Configure and secure a MySQL Database.
 28. Set up a Application Server (e.g., Apache Tomcat)
 29. create a service file for Apache Tomcat.(Should execute by using systemctl command)
 30. Print specific columns from a delimited file.
 31. Filter and print lines based on a specific pattern or condition.
 32. Calculate and print the average, sum, or other statistics of a column.
 33. Perform string manipulation, such as extracting substrings or changing case.
 34. Count the occurrences of a specific pattern in a file.
 35. Sort lines based on a specific field or column.
 36. Merge multiple files based on a common field or column.
 37. Substitute text in a file using search and replace.
 38. Delete specific lines based on a pattern or line number.
 39. Append or insert text before or after a specific pattern or line.
 40. Print only specific lines from a file.
 41. Copy file from linux to windows machine
 42. 5 use cases for AWK and 5 use cases for sed
-
-

DAY:02

TOPIC: WEB,APP SERVER AND HA PROXY CONFIGURATION

1. Install nginx and run nginx on port number 81.
 2. Deploy a sample index.html file on nginx.
 3. Install Apache and run Apache on port number 82
 4. Deploy a sample index.html file on Apache.
 5. Install Apache tomcat on port number 8082
 6. Deploy a sample app on webapps
 7. Create a tomcat.service file for tomcat.
 8. Configure HA Proxy server
-

DAY: 03

TOPIC: BASH SCRIPT AND CRON JOBS

Tasks to Execute

1. Execute all scripts from the **devops-cloud-documents-channel**.

Bash Script Tasks

1. Create a bash script to **check if a directory is available or not**.
2. Create a bash script to **create multiple files**.
3. Create a bash script to **take a backup of a directory**.
4. Create a bash script to **install Nginx on an EC2 server**.
5. Create a bash script to **install Apache Tomcat on an EC2 server**.
6. Create a bash script to **check if the Nginx service is running**, if not running then script should start the service.
7. Create a bash script for a **calculator**.
8. Create a bash script to **check if a directory exists**, if not then create a directory.
9. Create a bash script to **delete the last 3 lines of a file**.
10. Bash script to monitor cpu and if it is more than 80% then send email notification.
11. Bash script to monitor disk space and if it is more than 80% then send email notification.
12. Bash script to monitor memory and if it is more than 80% then send email notification.

Exercise: Crontab Entries

1. April 5th Midnight
 2. 5th of Every November, January, June **if it is a Thursday**
 3. At **05 and 27th minutes** of **9, 10, 11** hours every day
 4. **34 minutes** of **9th hour** on **15th August**
 5. Every midnight
 6. Every Weekend (**Saturday night 11:59 PM**)
 7. After every reboot
-

DAY 04

TOPIC : GITH & GITHUB TASKS

1. Install git.
 2. Create a repo in github with README.md and .ignore file.
 3. Clone the created repo to local.
 4. Create two files in local repo.
 5. Commit two files and push to central Repository.
 6. Create a branch in local and create a sample file and push to central.
 7. Create a branch in github and clone that to local.
 8. Merge the created branch with master in git local.
 9. Merge the created branch with master in github by sending a pull request.
 10. Create a file in local and send that to branch in github.
 11. Clone only a branch from github to local.
 12. Create a file with all passwords and make that untrackable with git.
 13. Make a commit and make that commit reset without saving changes.
 14. Revert a committed commit to the older version.
 15. Push a file to stash without saving the changes and work on another file.
 16. Undo the stash file and start working on that again.
 17. Generate a ssh-keygen and configure into github.
 18. Configure webhooks to github.
 19. Basic understanding of .git file.
 20. Check all the logs of git.
 21. Rename the commit message.
 22. Merge multiple commits into single commit.
-

TOPIC: GITH & GITHUB

CHALLENGES

Git Challenges

1. Resolve Merge Conflicts
 - Create a merge conflict intentionally (two users editing the same line).
 - Resolve the conflict and push the changes.
2. Recover Deleted Branch
 - Delete a local branch and then recover it using the reflog.
3. Undo Wrong Push
 - Push a wrong commit to GitHub, then undo it without losing history.
4. Amend a Commit
 - Make a commit, then add a missing file to it using `git commit --amend`.
5. Cherry-pick a Commit
 - Take a specific commit from one branch and apply it to another branch.
6. Interactive Rebase
 - Reorder and squash multiple commits into a single clean commit.
7. Tagging & Release
 - Create a version tag (v1.0), push it to GitHub, then delete and restore it.
8. Clone with Sparse Checkout
 - Clone only a subdirectory of a repo using sparse checkout.
9. Reset vs Revert Challenge
 - Demonstrate the difference between `git reset --hard` and `git revert` in a repo.
10. Detached HEAD Challenge

- Checkout a specific commit (detached HEAD state) and create a new branch from it.
11. Git Hooks Challenge
 - Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).
 12. Squash Merge vs Rebase Merge
 - Show the difference between squash merge and rebase merge with evidence.
 13. Fork & Pull Request Workflow
 - Fork a repo, make a change, and submit a pull request to the original repo.
 14. Recover Lost Commit
 - Commit something, reset hard, and then recover it using `git reflog`.
-

TOPIC: EC2

1. Launch one EC2 using Amazon Linux 2 image and add a script in user data to install Apache.
 2. Launch one EC2 using Ubuntu image and add a script in user data to install Nginx.
 3. Launch one Windows server and install Tomcat on Windows.
 4. Take a snapshot of the instance created in Task 1.
 5. Assign passwordless authentication for the EC2 created in Task 2.
 6. Launch any EC2 using the spot purchasing option.
 7. Enable termination policy on the EC2 created in Task 2.
 8. Launch one EC2 using AWS CLI.
-

TOPIC: Task on VPC01

1. Create VPC with 2 private and 2 public subnets.
 2. Enable DNS Hostname in VPC.
 3. Enable Auto Assign Public IP in 2 public subnets.
 4. Add 2 private subnets in private route table.
 5. Add 2 public subnets in public route table.
 6. Public route table will have the routes to internet and local.
 7. Create EC2 in public subnet with t2.micro and install PHP.
 8. Configure NAT gateway in public subnet and connect to private instance.
 9. Install Apache Tomcat in private EC2 and deploy a sample app.
 10. Configure VPC flow logs and store the logs in S3 and CloudWatch
-

Task on VPC02

1. Create one VPC, with 1 public subnet and 1 private subnet.
2. Enable VPC peering for cross-region.
3. Enable VPC peering for cross-account (you can collaborate with your friend to do this task).
4. Set up a VPC Transit Gateway.

5. Set up a VPC Endpoint.

Challenges of vpc

Scenario:

A large organization is migrating its on-premises infrastructure to the AWS cloud. The organization's architecture involves multiple VPCs for different departments and applications, each requiring secure communication with centralized services and external resources.

The IT team needs to design and implement a scalable and efficient network architecture to accommodate the organization's growth and ensure robust connectivity between VPCs and external services.

Objectives:

- Design and deploy a scalable network architecture using AWS Transit Gateway to simplify network connectivity between multiple VPCs.
 - Configure VPC endpoints to securely access AWS services without internet gateways or NAT gateways, ensuring data privacy and minimizing exposure to external threats.
-

S3

- @channel Task on S3
 - Notes: Create a Jira ticket for this assignment, work on the tasks listed below, and submit the task completion document by EOD with clear steps and required evidence (outputs/screenshots)
 - 1. Create an S3 bucket and upload some objects to S3..
 - 2. Deploy a static website in the S3 bucket..
 - 3. Enable cross-region replication on S3 buckets..
 - 4. Configure a bucket policy so only the Admin user can see the objects of the S3 bucket..
 - 5. Set up lifecycle policies to automatically transition or delete objects based on specific criteria..
 - 6. Push some objects to S3 using the AWS CLI..
7. Write a Bash script to create an S3 bucket..
8. Upload a 1 GB file to S3 using the CLI.
-

1. Configure VPC peering in cross regions.
2. Purchase one domain from GoDaddy.
3. Deploy static website in S3.
4. Create a CDN and attach one SSL certificate.
5. Create a Route 53 hosted zone and map the domain with the CDN.
6. Update the index.html in the S3 bucket and ensure the updated file is accessible using the domain name.
7. Share the domain name in Slack to test the connectivity.

Task on CloudTrail & CloudWatch

1. Enable CloudTrail monitoring and store the events in S3 and CloudWatch log events.
 2. Enable SNS for CloudTrail to send alerts via email.
 3. Configure CloudWatch monitoring and record the CPU utilization and other metrics of EC2.
 4. Create one alarm to send an alert to email if the CPU utilization is more than 70 percent.
 5. Create a Dashboard and monitor the Tomcat service whether it is running or not and send the alert.
 6. Create a Dashboard and monitor the Nginx service to send the alert if Nginx is not running.
-

Task for today ASG

1. Create one VPC in N. Virginia region.
2. Create two subnets: one public subnet and one private subnet.
3. Attach an IGW to the VPC.
4. Create one public route table (RT) and one private route table.
5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.
6. Create two instances, one in the public subnet and one in the private subnet.
7. Deploy Apache server on both EC2 instances with a sample index.html file.
8. Create one application load balancer and attach it to both EC2 instances.
9. Store application load balancer logs in S3.
10. Store the VPC flow logs in a CloudWatch log group.
11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.
-

Task on Load Balancers

1. Configure Classic Load balancer.
 2. Configure Application Load balancer.
 3. Configure Network Load balancer.
 4. Attach SSL for application load balancer.
 5. Map Application load balancer to R53.
 6. Push the application load balancer logs to S3.
-

Task on RDS

1. Create MariaDB DB on EC2.
 2. Insert some dummy data.
 3. Take the backup of dummy data on EC2.
 4. Launch MariaDB RDS instance.
 5. Migrate database from EC2 to RDS.
 6. Install MySQL DB on EC2.
 7. Launch MySQL RDS image.
 8. Configure Multi-AZ.
 9. Take backup of DB and restore the DB.
 10. Create read replica.
-

Start working on Jenkins-01.

1. Install Jenkins and run Jenkins on port number 8081.
 2. Secure Jenkins server
 3. Create users called DevOps, Testing in Jenkins with Limited access.
 4. Configure labels and restrict the jobs to execute based on label only.
 5. Create Three sample jobs using the below URL.
 - a. **Source Code:** https://github.com/betawins/Techie_horizon_Login_app.git
 6. Create one Jenkins job using GitHub Private repository..
-

Start working on Jenkins-02.

1. Configure 2 slave machines in the Jenkins master.
 2. Configure webhooks to the Jenkins job.
 3. Configure **Poll SCM** and **Build Periodically** options in the Jenkins job.
 4. Take a backup of the Jenkins server using a **bash-script**.
 5. Take a backup of Jenkins using the **ThinBackup** plugin.
 6. Set up a new Jenkins server and restore the backup taken in task 4.
-